

Reading Log

A private, reflection-first reading journal with shareable recommendation cards.

This document records why Reading Log was selected over the four alternatives, what v1 is and is not, how it is architected, and the day-by-day path to an App Store submission on September 14. It supersedes the five-concept brief for all planning purposes.

15

DAYS TO SUBMIT

1

PLATFORM AT LAUNCH

0

BACKEND SERVICES

Owner Nick Perry

Stack Flutter, iOS first, Android deferred

Window August 30 to September 17, 2026

Submit September 14 · **Target live** September 17

Status Scope locked September 1

SECTION ONE

The decision and the reasoning behind it

Why Reading Log, and why the other four were set aside.

The constraint that decided it

The original brief ranked five concepts against a thirty-day, iOS-only window with a day-24 submission target. Two things changed after that ranking was written. The build window compressed to roughly fifteen working days, and the implementation moved to Flutter with an intended dual-platform launch.

Both changes penalize concepts unevenly, and that is what produced the answer. Reading Log is the only concept in the set whose difficulty is essentially unchanged by either constraint.

CONCEPT	EFFECT OF THE FLUTTER MOVE	EFFECT OF THE COMPRESSED WINDOW
City Fog Map	Map tiling, fog overlay rendering, and location permissions all need parallel platform work. The single cheapest concept on iOS is no longer the cheapest.	Survivable, but the retention problem the original brief flagged is unaddressed and unaddressable in fifteen days.
Monthly Scrapbook	PhotoKit has no clean cross-platform equivalent. Photo library access, permissions, and thumbnail performance become two implementations.	Fatal. The brief was explicit that design quality is the product, and design quality is exactly what a halved timeline destroys first.
Reading Log	Effectively none. Open Library is a plain HTTP JSON API with no key. Nothing in the core loop touches a platform-specific framework.	Absorbable through scope reduction without damaging the premise. The core loop is small.
Daily Briefing	Severe. WeatherKit was the entire reason this ranked as high as fourth. Removing it means sourcing, paying for, and integrating a third-party forecast API, plus a widget on two platforms.	Fatal. It was already rated tight at thirty days with the most surface area of the five.
Household Systems	Neutral. The build was never the problem.	Fatal. Twenty of the thirty days were content authoring, and content does not compress.

What is actually good about it

- **It has a reason to exist.** Goodreads and Letterboxd optimize for ratings, shelf counts, and social feeds. Nobody serves the person who wants to remember why a book mattered. That is a real gap, not a positioning exercise.

- **The growth loop is free and built in.** The shareable recommendation card puts the app in front of another reader in the most trusted context available, a friend saying here is why you should read this. No marketing budget is required for it to work.
- **No backend, no accounts, no recurring cost.** Open Library supplies metadata and covers with no API key. Everything else lives on the device. This is the single largest driver of whether anything ships in a short window.
- **Search intent is genuine.** People type reading journal and book tracker. City Fog Map had nothing comparable.
- **The audience talks about its tools.** People who keep reading records are unusually vocal about how they keep them.

What we are accepting as known weaknesses

- **Low open frequency.** People finish books slowly. This app will never post daily-active numbers, and the App Store algorithm mildly rewards apps that do. Accepted.
- **Entrenched incumbents.** Goodreads has scale and Letterboxd has taste. We are not competing with either on breadth. We are competing on one narrow thing they both deprioritize.
- **Moderate screenshot appeal.** A reading journal is list-shaped, and list-shaped apps photograph worse than map-shaped ones. The recommendation card is the mitigation, and it needs to carry the first screenshot.

The thing that can actually kill this

The reflection flow is the entire product. If marking a book finished is one tap and writing about it is buried behind another screen, every user will take the tap and we will have shipped a worse Goodreads with no reason to exist. Reflection must be the path of least resistance, not an option alongside it. Equally, a blank text box intimidates people into writing nothing, so the prompts have to be specific enough to answer without effort. This is the one part of the build where the schedule bends rather than the quality.

SECTION TWO

Scope, locked

Frozen September 1. Anything not on the left-hand list is out until v1 is live.

IN V1

- Add a book via Open Library search
- Manual entry fallback for books the search misses
- Status: reading, finished, abandoned
- Lightweight progress logging
- **Reflection prompts on finishing.** The core of the product
- Reflection also available on abandoning, which is a differentiator nobody offers
- Shareable recommendation card, rendered to an image
- Local-only storage, no account, no server
- Onboarding of three screens or fewer

CUT FROM V1

- Quote and passage capture (was in the original scope, cut for time)
- Year in review (was in scope, cut for time, revisit in December)
- ISBN barcode scanning
- All social and discussion features
- Recommendation engine
- Reading challenges and numeric goals
- Film, TV, and music tracking
- Accounts, sync, and cloud backup
- Android release (deferred, see Section Five)

On the auth and backend question

The original plan reserved September 6 to 8 for backend, auth, and storage. The entire argument for this concept fitting a compressed window is that it is local-only with no accounts. Adding auth reintroduces the exact cost that was cut, in half the original time budget, for a feature no user has asked for in a private single-device journal. **Recommendation: cut auth from v1 entirely.** Ship local storage with a manual JSON export and import, which takes hours rather than days and covers the one real user need, which is not losing their data when they change phones. If auth stays in, it is the first thing to cut when the schedule slips, and the schedule will slip.

Website roadmap, not in-app

Apple rejects apps that advertise unreleased features in the UI. Film, TV, and music tracking, discussion features, and year in review all belong on the marketing site behind an email signup, and nowhere in the binary. No coming soon labels, no greyed-out tabs, no placeholder screens.

SECTION THREE

Architecture and data model

Shape of the app

Four surfaces, no more. Every additional tab is a screenshot that has to be designed, a state that has to be tested, and a decision the user has to make before they get value.

Shelf	Currently reading, then finished, then abandoned
↳ Book	Cover, status, progress, reflections
↳ Reflect	The finishing flow. Prompted, sequential, skippable
↳ Card	Rendered image, share sheet
Add	Open Library search, manual fallback
Settings	Export, import, about, privacy

Data model

The lesson carried over from the Household Systems analysis applies here in a smaller form. Model the reflection as a first-class record attached to a reading session, not as a set of columns on the book row. A person can read the same book twice and think something different the second time, and a schema that assumes one book equals one opinion cannot represent that later without a migration.

ENTITY	FIELDS	WHY IT IS SEPARATE
Book	id, title, author, cover ref, Open Library key, published year, source (api or manual)	Immutable reference data. Cacheable, replaceable.
ReadingSession	id, book id, status, started, ended, progress, abandoned reason	One book can have many sessions. Rereads are real.
Reflection	id, session id, prompt key, response text, created	One row per prompt answered. Adding a prompt later is data, not a schema change.
Card	session id, theme, selected reflection ids, generated at	Sharing is a view over reflections, never a copy of them.

Technical decisions, made now so they are not made at 2am on September 10

- **Local persistence:** a single embedded database with a versioned migration path from the first commit. Not shared preferences, not loose JSON files.
- **Covers:** fetched from Open Library, cached to disk, with a designed typographic fallback for books that have no cover. The fallback will be used more often than expected and cannot look broken.

- **Offline:** everything except search works with no network. Search fails with a message and a manual entry button, never with a spinner that never resolves.
- **Card rendering:** render the card widget to an image on device. No server-side image generation, no headless rendering service.
- **Cross-platform discipline:** build and run against the Android simulator daily even though Android is not shipping. Platform divergence caught today is free. Caught in October it is a week.

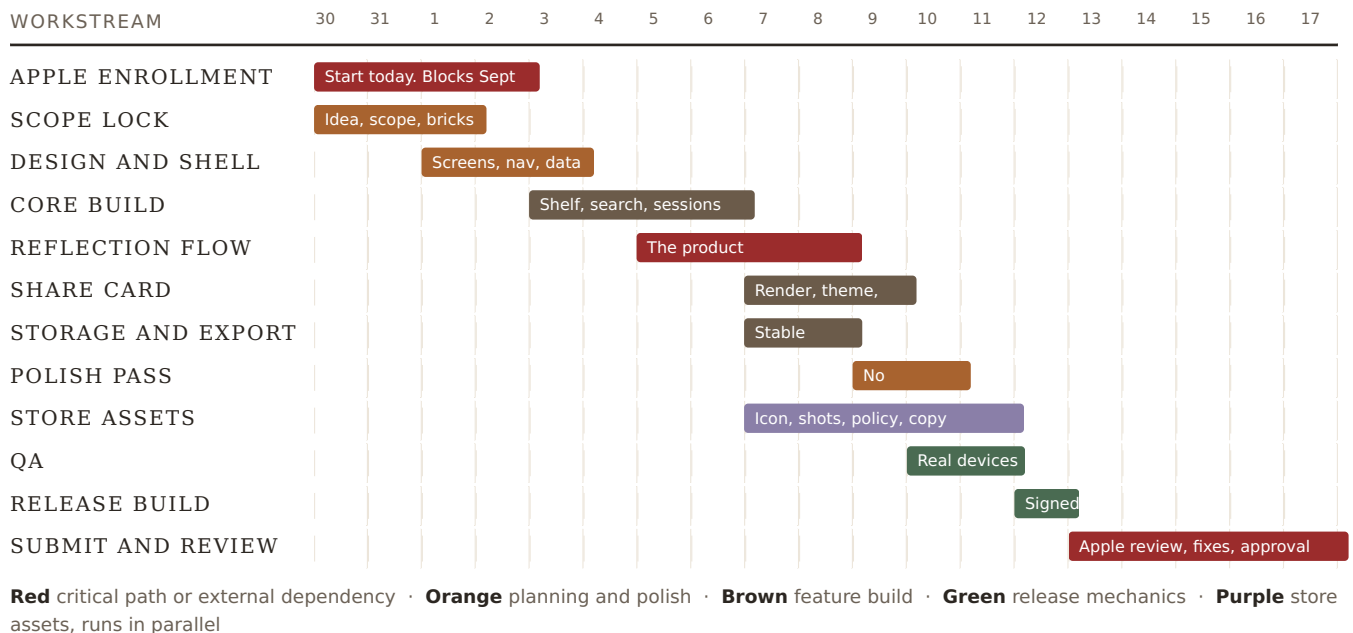
Reflection prompts, first draft

Three prompts on finishing, each skippable, each with a one-line example beneath it so the box is never blank. Suggested set: **What stayed with you?** · **Who should read this, and why them?** · **What did it change, if anything?** The third is optional and should be visually lighter than the first two. Prompts are stored by key, so the wording can be revised in an update without touching stored responses.

SECTION FOUR

Visual roadmap

August 30 to September 17. Submission on day 15.



The two dates that are not yours to move

- **Apple Developer enrollment.** It can take days, not hours, and it sits directly under the September 13 signed build. If it is not already active, it is the first action of this project, ahead of any code.
- **Apple review.** Typically under a day now, but historically one to three and always unpredictable. Submitting on the 14th for a target of the 17th leaves room for exactly one rejection round. There is no second round in this window.

Where the original schedule was too tight

QA closed on the 11th and store metadata started on the 12th, with no slack between them. Store assets are roughly two days of work that is almost never budgeted, and they do not depend on QA passing. In the plan above they start on the 8th and run in parallel. That change alone is what makes the 14th realistic.

SECTION FIVE

Day by day

DATE	MILESTONE	WHAT DONE MEANS
Aug 30	Apple enrollment started	Payment submitted. Nothing else on this list can complete without it.
Sept 1	Scope locked	Concept, v1 feature list, and Flutter package choices written down and closed. No additions after this date without an equivalent cut.
Sept 3	Screen map, navigation, data model, shell	Four surfaces navigable with dummy data. Migration-capable schema in place. Reflection prompt wording drafted.
Sept 6	Core working on iPhone	Add a book from Open Library, set status, log progress, see the shelf. Runs on a real device, not just a simulator. Android simulator still building clean.
Sept 8	Storage stable, assets started	Persistence across cold starts and app upgrades. Export and import working. Reflection flow complete end to end. Icon and screenshot direction chosen.
Sept 10	Polish pass done	No placeholder text, no lorem ipsum, no default icon, no broken empty states. Every screen has a considered empty state, since a new user sees nothing but empty states.
Sept 11	QA pass, blockers closed	Real device, airplane mode, small screen, largest dynamic type, fresh install, upgrade over previous build. Blocker list closed or explicitly cut in writing.
Sept 12	Store metadata ready	Icon, screenshots at required sizes, privacy policy URL live, support URL live, description, keyword field, App Privacy questionnaire complete.
Sept 13	Signed release build	Archived, uploaded, and processed in App Store Connect. Installed from TestFlight on a device that has never had a debug build.
Sept 14	Submit to Apple	Submitted for review.
Sept 15 to 16	Review response	Any rejection answered same day. Only rejection-blocking fixes. No opportunistic changes.
Sept 17	Approved or publish-ready	Live, or approved and held for a chosen release date.

Android, deferred

Android is postponed from this window and gets its own date after iOS is live. The reason is external and not schedulable around.

Google Play closed testing requirement

Personal Play Console accounts created after November 13, 2023 must run a closed test with at least twelve testers opted in continuously for at least fourteen days before they can apply for production access. The requirement applies per app, so testers from a previous project do not carry their opt-ins over, and the fourteen-day counter restarts if the count drops below twelve. Google also assesses whether those testers genuinely used the app. Organization accounts and personal accounts created before that date are exempt.

First action: confirm which account type you hold. If exempt, Android can rejoin the schedule immediately after iOS submission. If not, the practical path is to start closed testing on the first installable build rather than the finished one, since the clock runs on a testable build, not a polished one, and can run concurrently with polish and review. Recruit fifteen rather than twelve to absorb drop-outs.

SECTION SIX

Risk register and cut order

RISK	SEVERITY	RESPONSE
Reflection flow feels like a chore	CRITICAL	Test it on three real people before September 10 with a book they actually finished. If they skip all three prompts, the prompts are wrong, not the users. Rewrite rather than ship.
Apple enrollment not complete by Sept 13	CRITICAL	Unrecoverable within this window. Start August 30. Check status daily until active.
Scope creep past September 1	HIGH	One in, one out. Any addition requires a written cut of equal size on the same day.
Open Library search quality or downtime	HIGH	Manual entry fallback is in v1 scope specifically for this. Search failure must never be a dead end.
Store assets underestimated	HIGH	Started September 8 in parallel rather than after QA. Historically two full days.
Rejection for unreleased feature promotion	MEDIUM	No roadmap language anywhere in the binary. Verify during the September 10 polish pass, not after submission.
Card rendering differs across screen sizes	MEDIUM	Fixed output dimensions independent of device. Verified on smallest and largest devices during September 11 QA.

Cut order, decided in advance

If September 10 arrives and the app is not polish-ready, cut in this order without reopening the debate. Deciding this now is what prevents the wrong thing being cut under pressure at midnight.

1. Auth and any remote storage, if it survived Section Two at all
2. Progress logging, keeping only the three statuses
3. Card theming, shipping one well-made theme instead of several
4. Abandon status, keeping reading and finished
5. Manual entry, if and only if Open Library coverage tested well

Never cut: the reflection prompts, or the share card. Those two are the product. An app that ships without them is not a smaller version of this app, it is a different and worse one that already exists.

Definition of done for v1

A person can add a book they are reading, mark it finished, answer three prompts about it in under ninety seconds, and text a friend a card that makes that friend want to read it. Everything else in this document exists to protect that sentence.